perf_analyzer 工具及火焰图原理

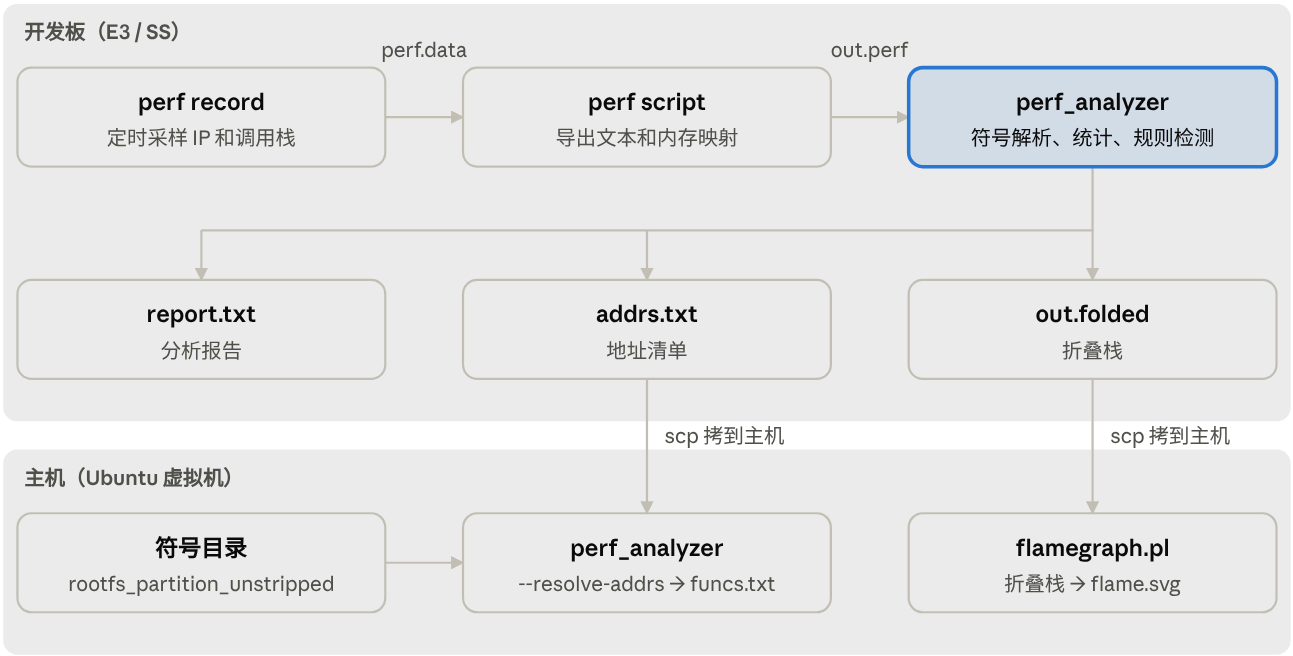
Oct 13, 2026 · @john.he

1. 概述与整体数据流

perf_analyzer 解决的是这样一个问题：嵌入式板子上的程序被 strip 过、板子上的 perf 又不能读取符号表，perf 采到的只是一串地址；人工看火焰图虽然能找到热点，但每次都要重复同样的判断。perf_analyzer 把"地址 → 函数名 → 统计 → 判断问题模式"这条链路做成了一个工具，在主机和板子上都能运行。

整个流程分三段：perf 负责**采样**，perf_analyzer 负责**解析和分析**，FlameGraph 脚本负责**画图**。符号表只在主机上的未 strip 文件里才完整，所以需要函数名时，板子上先输出地址，再到主机上换算。

板子上采集和分析，主机上换算符号和画图



perf_analyzer 数据流 · 板子 3 步，主机 2 步

板子上的 perf_analyzer 一次输出三个文件：`report.txt` 可以直接阅读，`addrs.txt` 和 `out.folded` 拷到主机后分别换算成函数名、画成火焰图。本文按数据流的顺序依次说明每一步的原理：采样（第 2 节）、调用栈回溯（第 3 节）、符号解析（第 4 节）、perf_analyzer 的分析方法（第 5 节）、火焰图（第 6 节），最后说明误差和局限（第 7 节）。

2. perf 采样原理

2.1 定时采样

`perf record -e cpu-clock -F 199` 让内核用高精度定时器每秒中断 199 次。每次中断时，如果被监控的进程正在 CPU 上运行，内核就记录一个**样本**：

字段	含义
comm、pid/tid	线程名、进程号/线程号，用于按线程统计
time、period	时间戳和距上次采样的时长（cpu-clock 下单位为纳秒，约 5,025,125 ns = 1/199 秒）
ip	中断发生时 CPU 正在执行的指令地址，也就是"采样点"
callchain	从采样点往上的调用栈：先是内核部分，再是用户态部分

采样是统计意义上的测量：某个函数出现在样本里的比例，近似等于它占用 CPU 时间的比例。**CPU 时间 ≈ 样本数 × period**，例如 689 个样本 × 5.025 ms ≈ 3.46 秒，再除以采集时长 120 秒，得到平均占用单核约 2.9%。

线程睡眠、等待锁或 IO 时不在 CPU 上，定时器中断时不会为它记录样本，所以 perf 测量的是**on-CPU 时间**。

2.2 为什么用 cpu-clock

`cpu-clock` 是由内核定时器驱动的软件事件，任何平台都能用。硬件事件 `cycles` 依赖 CPU 的性能监控单元（PMU）和它的中断配置，AM335x 等平台的 BSP 经常没有配好 PMU 中断，采样结果会集中在少数几个函数上甚至无法使用。

2.3 内存映射和内核符号

样本里只有地址。要知道地址属于哪个文件，还需要进程的**内存映射**：

- 用 `-p` 附加到已运行的进程时，perf 读取 `/proc/<PID>/maps`，把已有的映射"补录"成 MMAP 事件；`--proc-map-timeout` 控制补录的时间上限，进程映射多时要放宽，否则样本无法归属到文件。
- 采集期间新加载的库，由内核实时产生 MMAP2 事件。

- `perf script --show-mmap-events` 把这些事件输出成文本，perf_analyzer 靠它们换算地址。

内核函数名来自 `/proc/kallsyms`。普通情况下内核会隐藏符号地址，所以采集前要执行 `echo 0 > /proc/sys/kernel/kptr_restrict`。

3. 调用栈回溯原理

3.1 帧指针链

`--call-graph fp` 表示用**帧指针**回溯用户态调用栈。每个函数被调用时在栈上保存一个"栈帧记录"，里面有上一层的帧指针和返回地址；帧指针寄存器指向当前函数的记录。内核在采样时从帧指针寄存器出发，读出返回地址，再跳到上一层的记录，一层层往上走，得到调用链。

```
采样点 ip ──► 当前函数
帧指针 fp ──► [ 上一层 fp | 返回地址 ] ──► 调用者
                  |
                  ▼
              [ 再上一层 fp | 返回地址 ] ──► 调用者的调用者 ……
```

这种方式开销极小，但要求每一层函数都按约定保存帧记录，**任何一层不符合约定，链就在那里断开**。

3.2 为什么需要 -mapcs-frame

32 位 ARM 上有两种帧记录格式，内核只认其中一种：

编译选项	函数入口保存的内容	内核能否回溯
`-fno-omit-frame-pointer`（GCC 默认格式）	`push {fp, lr}`，fp 指向保存的 lr	不能，格式和内核期望的不一致
`-fno-omit-frame-pointer -mapcs-frame`（APCS 格式）	`push {fp, ip, lr, pc}`，fp 指向保存的 pc	能，内核按 `{fp, sp, lr}` 的帧尾结构读取

32 位 ARM 内核和 64 位 ARM 内核（回溯 32 位程序时）都按 APCS 格式解析用户栈，所以工程里 `PROFILE=1` 真正起作用的就是 `-mapcs-frame`。实测 E3 上没有它时 95.9% 的样本只有一层栈帧，加上后降到 42.4%。它让每个函数入口和出口多一两条指令，ApplicationServices 代码段增加约 1%，SS 根分区只多占约 0.2 MB。

3.3 剩余的断栈从哪里来

加上 `-mapcs-frame` 后仍有约 40% 的样本只有一层用户态栈帧，几乎全部来自**采样点落在系统库里**的样本：libc、libstdc++、libpthread 等来自板子的根文件系统，不是按 APCS 格式编译的（可能还是 Thumb-2 指令，用 r7 而不是 fp 作帧指针）。采样点在这些库内部时，帧指针寄存器里不是有效的帧记录，回溯在这一层就停止，调用者丢失。这也是报告中 gettid、clock_gettime 的部分调用者显示为"未知"的原因。你们自己的代码都带 APCS 栈帧，采样点落在业务代码里时调用链是完整的。

3.4 栈底的 [perf-N.map] 帧

回溯走到线程最底层时，最后读到的"返回地址"往往是栈上的无效值。ApplicationServices 的线程栈是可执行的匿名内存（程序被链接成可执行栈，`GNU_STACK` 带 E 标志），无效值恰好落在这里，perf 把匿名可执行内存当作 JIT 代码，标成 `[perf-<pid>.map]` 或 `[JIT]`，`perf script` 还会提示找不到 `/tmp/perf-<pid>.map`。这一帧没有意义，perf_analyzer 默认把栈底连续的这类帧去掉。

3.5 返回地址

除了采样点本身，调用链上其他层记录的都是**返回地址**，也就是调用指令的下一条指令。换算成函数名时一般落在调用者函数内部；极少数情况下（调用位于函数最后一条指令，例如调用不返回的函数）会落到相邻的下一个函数。判断热点时以"自身耗时"为准。

4. 符号解析原理

4.1 ELF 文件里的符号

程序和动态库都是 ELF 格式，和函数名相关的有两张表：

表	内容	什么时候存在
`.symtab`（符号表）	所有函数，包括 static 函数和 C++ 类的内部函数	普通编译就有，不需要 `-g`；`strip --strip-all` 会删掉
`.dynsym`（动态符号表）	只有对外导出、供动态链接使用的函数	动态库和动态链接的程序都有，strip 也不会删
`.debug_*`（调试信息）	源文件、行号、变量类型	只有 `-g` 编译才有；`strip --strip-debug` 删掉它但保留 `.symtab`

这解释了板子上的现象：固件用 `--strip-all` 打包，ApplicationServices 只剩 `.dynsym`，内部函数无法解析；但自研库导出的函数（如 `Json::Reader::parse`、`PersistentStoreService::managerTask()`）和 libc 的导出函数（`gettid`、`malloc`）仍然能显示名字。`grep -c '\.symtab' <文件>` 输出 0 就说明符号表已被删除。

4.2 地址换算

样本里是**运行时地址**，符号表里是**链接地址**，两者之间要经过两步换算：

```
文件偏移 = 运行时地址 − 映射起始地址 + 映射的文件偏移(pgoff)      ← 来自 MMAP 事件
链接地址 = 文件偏移 − 所在 PT_LOAD 段的 p_offset + 该段的 p_vaddr   ← 来自 ELF 程序头
```

然后在按地址排序的符号表里二分查找：找到起始地址不大于链接地址、且链接地址落在其大小范围内的那个函数。

- **PIE 程序和动态库**：每次加载的位置不同，必须依赖 MMAP 事件，这就是 `perf script` 必须加 `--show-mmap-events` 的原因。
- **非 PIE 程序**：运行地址就是链接地址，没有 MMAP 事件也能解析。
- **Thumb 指令**：ARM 上 Thumb 函数在符号表里的地址最低位为 1，查找前要清掉。
- **C++ 名字**：符号表里是 `_ZN9Scheduler14ScheduleUpdateEi` 这样的修饰名，经 demangle 还原为 `Scheduler::ScheduleUpdate(int)`。

4.3 为什么用未 strip 的副本解析运行中的 strip 版本

`strip` 只删除不加载到内存的段（符号表、调试信息），代码段和数据段的内容、文件偏移、链接地址都保持不变。所以对同一次编译的产物，在板子上算出的链接地址，可以直接拿到主机上未 strip 的 `rootfs_partition_unstripped` 里查函数名。这就是"板子记地址、主机换算"能成立的前提。

4.4 版本一致性

换算只依赖地址，不会检查两个文件是不是同一次编译。如果符号目录和板子上的固件不是同一次编译，换算照样能得出"看起来合理"的函数名，但都是错的：之前用 E3 的旧文件解析 SS 程序时，出现了 `validateSetPointValues → n_GetMask → changeBACnetIPPort` 这样业务上不可能的调用链，就是这个原因。

板子上的 perf 没有 libelf，读不到 build-id，所以用 md5 核对：固件就是用 `strip --strip-all` 打包的，在主机上对未 strip 文件做一次同样的 strip，md5 应该与板子上的完全一致。

5. perf_analyzer 的实现原理

5.1 模块结构

模块	职责
`input_parser`	识别输入格式，逐行解析 `perf script` 的样本头、栈帧行和 MMAP 事件，或解析折叠栈
`elf_symbolizer`	维护内存映射和 ELF 文件缓存，完成地址换算和符号查找（第 4 节）
`dataset`	数据模型：函数名字符串池、调用栈、统计用的汇总信息
`symbol_rules`	函数分类规则：内核/库/业务、系统调用入口、异常入口、栈底异常帧、内核架构线索
`analyzer`	统计、各项检测、规则引擎和报告输出
`addr_resolver`	主机上把地址清单换算成函数名

5.2 数据模型

每个样本被整理成一条**从根到叶**的调用栈：第 0 层是线程名，最后一层是采样点所在函数，每一帧另外记录是否为内核帧。函数名通过字符串池映射成整数 id，几万个样本、几十万个栈帧的统计都只做整数运算。

输入格式自动识别：有以空白开头的栈帧行的是 `perf script` 输出，每行以"分号分隔的栈 + 数字"构成的是折叠栈。折叠栈里的计数如果都是同一个大数的整数倍（新版 stackcollapse 会累加 period），自动换算回样本数。

5.3 统计口径

指标	计算方法
自身耗时	采样点（栈的最后一层）落在该函数的样本数
累计耗时	栈中任一层出现该函数的样本数，同一个样本只计一次（递归调用不会重复计数）
线程占比	第 0 层为该线程的样本数

指标	计算方法
CPU 时间	period 之和；没有 period 时用 样本数 × 采样频率

所有占比都除以全部样本数。"业务函数"是指既不是内核函数、也不是库函数的帧：`std::`、`__` 开头的运行时函数，常见 libc 函数名，以及 libc、libstdc++、libglib、libdbus 等系统库的地址帧，都归为库函数。

5.4 主要检测算法

栈底异常帧：第 1 层开始连续的 `[perf-N.map]`、`[JIT]`、`[unknown]` 帧被去掉，并统计受影响的样本比例。

系统调用判断：栈中出现系统调用入口函数（64 位内核的 `el0_svc`、`__arm64_sys_*`，32 位内核的 `vector_swi`、`sys_*` 等）就算作系统调用。32 位 ARM 内核的调用栈里通常看不到入口函数，所以还有一条补充规则：从用户态帧进入内核帧、且内核部分不经过中断或缺页入口（`__irq_usr`、`__dabt_usr`、`do_page_fault` 等），也算作系统调用。进入内核前最后一个用户态帧，就是发起系统调用的函数（如 `gettid`、`__select`）。

内核架构推断：内核帧中出现 `el0_*`、`__arm64_*` 判为 64 位 ARM，出现 `vector_swi`、`ret_fast_syscall`、`__irq_usr` 等判为 32 位 ARM，据此调整报告中的说明文字。

线程入口识别：如果某个业务函数在至少 3 个线程里、作为最底层的业务函数出现在该线程 25% 以上的样本中，就判为线程入口包装函数（如 `PosixTaskWrapper`、`libG4AppUtilities.so+0x125cc`）。它们的累计占比很高但不是热点，会在线程分布、调用路径和累计耗时中跳过。

最热调用路径：把一个线程的所有样本按业务函数建成调用树，从根开始每层选择累计样本最多的子节点往下走，直到下一层不足全部样本的 2%。得到的是"这个线程大部分时间沿着哪条路径执行"。

调用者查找：对规则命中的目标函数（如 `gettid`），在每个样本里从目标往根方向找最近的业务函数，作为调用者统计。回溯在系统库里中断时，调用者记为"未知"。

数据质量检查：样本少于 300 或 1000 个、只显示库名的采样点超过 5%、出现 `__fgetspent_r` 等不可能成为热点的 libc 函数超过 1%（libc 版本不匹配的标志）、栈底异常帧超过 30%、用户态只有一层的样本超过 25%，分别给出提示。

5.5 规则检测与优先级

每条规则是一组目标函数加一个阈值（`--min-pct`，默认 1%）。命中后统计它的累计占比、业务调用者和所在线程，并附上固定的优化建议：

规则	识别依据	改动难度
频繁调用 gettid	`gettid`	很小
频繁获取系统时间	`clock_gettime`、`gettimeofday`	小
64 位软件除法	`__udivmoddi4`、`__aeabi_uldivmod` 等	小
等待类调用占用 CPU	`select`、`poll`、`epoll_wait`、`nanosleep` 等	小
疑似反复排序	名字含 compare/cmp 的业务函数，且由库函数回调	中
字符串键 map 查找	字符串键的 `_Rb_tree` / `_Hashtable`，以及字符串比较和构造	中
内存分配频繁	`malloc`、`free`、`operator new` 等	中
锁和信号量	`sem_*`、`pthread_mutex_*`、`PosixSemTake` 等	中
JSON 解析	`Json::Reader::*`、`nlohmann::*` 等	中
系统调用整体占比偏高	系统调用中的样本超过 20%	汇总项

优化优先级按 **占比 ÷ 难度系数** 排序，难度系数为：很小 1、小 1.5、中 2.5。所以占比 5.5% 的 gettid（5.5 ÷ 1）会排在占比 7.4% 的锁和信号量（7.4 ÷ 2.5）前面：改动小、收益确定的项先做。

5.6 地址模式和地址清单

加 `--addr` 后，无法解析出函数名、但能换算出链接地址的帧显示为"文件名+0x链接地址"。这个地址与加载位置无关，同一次编译的产物在任何地方换算结果都相同。系统库的地址帧归为库函数，其他（程序本身和自研库）归为业务函数，所以报告的调用路径、累计耗时和规则调用者都能以地址形式给出。

`--addr-list` 把每个地址帧的板子路径、链接地址、自身样本和累计样本写成清单。主机上 `--resolve-addrs` 读取清单，在 `--symfs` 下的未 strip 文件中查函数名，输出"按地址明细"和"按函数汇总"两张表；汇总按"文件 + 函数"分组，同一函数的多个地址合并，累计占比为各地址之和（递归调用时偏大）。

6. 火焰图原理

6.1 折叠栈

火焰图的输入是**折叠栈**：每一行是一条从根到叶、用分号连接的调用栈，后面跟着这条栈出现的样本数。完全相同的栈合并成一行。perf_analyzer 的 `--folded-out` 输出的就是这种格式，第一层是线程名：

```
DataServices;dataServerTask;ScheduleUpdate;GetPropertyValue 3
DataServices;dataServerTask;ScheduleUpdate;getTicks 1
DataServices;dataServerTask;Json::parse 2
TimerServices;compareEvents 2
```

6.2 从折叠栈到火焰图

`flamegraph.pl` 把所有栈合并成一棵树：同一位置上名字相同的帧合并为一个方块，方块宽度等于经过它的样本数之和。上面 4 行（共 8 个样本）画出来是这样的（按每个样本 10 个字符宽示意）：

```
[GetPropertyValue 3          ][getTicks]
[ScheduleUpdate 4                      ][Json::parse 2     ]
[dataServerTask 6                                          ][compareEvents 2
]
[DataServices 6                                            ][TimerServices 2
]
[all 8
]
```

维度	含义
纵向	调用深度。下面是调用者，上面是被调用者；最底层是全部样本
宽度	经过该函数的样本数，也就是它的**累计耗时**
顶边	一个方块上方没有被子方块覆盖的部分，是采样点直接落在该函数里的样本，也就是**自身耗时**
横向顺序	同一层按函数名字母排序，**不代表时间先后**，左右位置没有意义

维度	含义
颜色	默认随机暖色，只为区分相邻方块；`--color=java` 配合 `_[k]` 后缀时，内核和用户态用不同颜色

6.3 怎么读火焰图

- **找宽的"平顶"**：上方没有子方块、又很宽的方块，就是自身耗时高的函数，是最直接的优化目标。例子里的 `GetPropertyValue` 平顶占 3/8。
- **找宽的"塔"**：底部很宽、往上分叉很多的方块（如 `dataServerTask`），说明它本身不耗时，耗时在它调用的函数里，要沿着塔往上找。
- **同一个函数出现在多个塔的顶上**（例如 `malloc`、`gettid` 被很多地方调用）：火焰图里它被分散成许多窄条，不容易看出总量。这时用右上角 Search 输入函数名，右下角会显示合计占比；或者用冰柱图。
- **栈很浅、大量方块直接贴在线程名上**：说明回溯在这一层断了（第 3.3 节），调用者看不到，不是程序只有一层调用。

6.4 变体

变体	生成方式	用途
冰柱图	`flamegraph.pl --reverse --inverted`	把栈倒过来，从采样点往调用者方向合并，被很多地方调用的底层函数（malloc、gettid、锁）会合并成一个宽方块
差分火焰图	`difffolded.pl 前.folded 后.folded | flamegraph.pl`	对比优化前后：宽度表示优化后的值，红色表示变多、蓝色表示变少
单线程火焰图	perf_analyzer 加 `--comm 线程名`	只看一个线程，避免被其他线程干扰

6.5 火焰图和 perf_analyzer 报告的关系

两者来自同一份折叠栈，是同一组数据的两种呈现：火焰图把**全部**调用关系一次画出来，适合人工探索和发现意料之外的热点；perf_analyzer 报告把其中有代表性的部分提取成数字（自身/累计排行、最热路径），并用规则把常见问题模式直接指出来。实际使用时先看报告确定方向，再到火焰图里沿着对应的路径细看。

7. 误差与局限

7.1 采样的统计误差

采样得到的占比是估计值。一个真实占比为 p 的函数，在 n 个样本中的估计误差（95% 置信区间）约为 $\pm 1.96 \times \sqrt{p(1-p)/n}$：

样本数	真实占比 5% 时的误差	真实占比 20% 时的误差
300	±2.5 个百分点	±4.5 个百分点
689	±1.6 个百分点	±3.0 个百分点
2,000	±1.0 个百分点	±1.8 个百分点
5,000	±0.6 个百分点	±1.1 个百分点
20,000	±0.3 个百分点	±0.6 个百分点

所以几百个样本时，2% 以下的项基本是噪声，两个相差一两个百分点的热点分不出先后。程序负载很低时（如 ApplicationServices 平均只占 2%～3%），提高频率或延长采集时间才能攒够样本，最好在业务繁忙或问题复现时采集。

7.2 其他局限

局限	影响	应对
只统计 on-CPU 时间	线程阻塞、等锁、等 IO 的时间不出现，"卡顿但 CPU 不高"的问题看不到	这类问题需要 off-CPU 分析（调度事件、锁等待统计），不在本工具范围内
系统库里断栈	采样点在 libc、libstdc++ 内部时调用者丢失（第 3.3 节）	关注业务代码部分；规则中"调用者未知"的比例就是这部分
内联函数	`-O2` 下被内联的小函数不会单独出现，耗时算到调用者头上	用 addr2line `-i` 配合 `-g` 编译的文件查看内联展开
尾调用优化	函数末尾直接跳转到另一个函数时，中间那一层不在栈上	调用链偶尔会"跳过"一层，属正常现象
返回地址	调用者一侧的地址偶尔落到相邻函数（第 3.5 节）	以自身耗时为准
符号版本	符号目录与运行的程序不一致时，函数名错位而且看起来合理（第 4.4 节）	每次分析前用 md5 核对

局限	影响	应对
规则是启发式的	规则只按函数名和比例判断，不理解业务逻辑；命中的规则是"值得检查"，不等于"一定有问题"	结合代码确认后再优化，优化后重新采集对比（差分火焰图）
采样本身的开销	每秒 199 次采样加帧指针回溯，对 CPU 的影响通常远小于 1%	采集结束后再在板子上运行 `perf script` 和 perf_analyzer，避免分析过程影响结果

7.3 结论的可信度怎么判断

读报告时先看第一部分"数据质量检查"：样本数是否足够、是否有 libc 版本不匹配的提示、用户态只有一层的比例有多高。这些都正常时，报告里占比较高、且在多次采集中稳定出现的热点和规则，才是值得投入优化的对象。